SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team ID : 109 Team Rojak

1. Introduction

1.1. Purpose

This System Analysis Documentation (SAD) highlights the technical scope, architecture decisions, data flows, and AI integration behind StockWise, which is an AI-powered inventory optimization system developed for UMHackathon 2026. The document serves as the authoritative technical reference for all stakeholders involved in the evaluation, development, and future extension of the platform.

StockWise addresses a critical gap in the Malaysian SME market by combining traditional inventory management capabilities with advanced AI reasoning. The system leverages Z AI's GLM (General Language Model) to analyze not only quantitative inventory data but also qualitative market signals such as currency exchange rate fluctuations and industry news, producing actionable recommendations with transparent reasoning chains.

The key elements covered in this document include:
(a) the overall system architecture and component interactions,
(b) detailed data flow diagrams illustrating how information moves between the frontend, API layer, database, and AI service,
(c) the AI model integration process including prompt construction, response parsing, and database persistence, and
(d) a comprehensive reference for developers and testers to understand the system's design rationale and technical decisions.

1.2. Background

Malaysian Small and Medium Enterprises (SMEs) form the backbone of the national economy, contributing over 38% of the country's GDP. However, many of these businesses continue to rely on spreadsheet-based inventory tracking or legacy systems that lack intelligent decision-making capabilities. This results in significant inefficiencies including overstocking, stockouts, missed procurement opportunities, and an inability to respond dynamically to external market conditions such as currency fluctuations and supply chain disruptions.

Traditional inventory management systems provide basic stock-level tracking and reorder point alerts but lack AI reasoning capabilities. They cannot incorporate multi-factor analysis that considers exchange rate volatility, market news sentiment, sales velocity trends, and supplier lead times simultaneously. This limitation forces business owners to make procurement decisions based on incomplete information, leading to suboptimal inventory positions and increased operational costs.

StockWise was conceived and built during UMHackathon 2026 as a full-stack Next.js application that bridges this gap. By integrating Z AI's GLM as a reasoning engine, StockWise transforms raw inventory data into intelligent, context-aware recommendations. The system was designed as a Minimum Viable Product demonstrating how large language models can be practically applied to supply chain optimization for Malaysian SMEs, with a focus on transparency (reasoning chains), actionability (priority-ranked suggestions), and strategic planning (scenario simulations).

1.3. Target Stakeholder

The following table outlines the key stakeholders of the StockWise system, their roles within the organization, and their primary expectations from the platform. Understanding these stakeholders is essential for ensuring that the system meets the diverse needs of its user base, from day-to-day warehouse operations to strategic business planning.

Stakeholders	**Roles**	**Expectations**
Warehouse Manager	Oversees daily warehouse operations, monitors stock levels, and coordinates with procurement teams to maintain optimal inventory levels.	Real-time visibility into stock levels across all products; automated AI-generated alerts for critical and low-stock items; actionable reorder recommendations with clear reasoning; ability to log inventory movements and adjustments directly within the system.
Procurement Officer	Responsible for supplier relationships, purchase order creation, and ensuring timely replenishment of inventory based on demand forecasts and market conditions.	Access to AI-generated recommendations with transparent reasoning chains showing why each suggestion was made; review of exchange rate impacts on procurement costs; integration of market news analysis into purchasing decisions; confidence in AI outputs through explainable reasoning.
Business Owner / CEO	Sets strategic direction for the company, makes high-level decisions about	Ability to run multi-scenario strategy simulations comparing different procurement approaches;

	inventory investment, cost optimization, and risk management strategies.	visualization of potential cost savings from AI recommendations; dashboard KPIs showing overall inventory health, risk exposure, and financial impact; data-driven insights for strategic planning.
Development Team	Responsible for maintaining the codebase, implementing new features, fixing bugs, and ensuring system performance and reliability.	Clear documentation of system architecture and data flows; well-structured, modular codebase with clean separation between frontend, API routes, and database layer; comprehensive understanding of AI integration patterns for extending GLM capabilities; reproducible development and deployment processes.
QA / Testing Team	Ensures software quality through systematic testing of features, data flows, AI outputs, and user interface consistency.	Detailed test cases covering all API endpoints and data flows; validation criteria for AI-generated recommendations including accuracy and relevance; clear understanding of expected system behavior under various scenarios; documentation of edge cases and error handling expectations.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. System Overview

The table below provides a high-level summary of the StockWise system's architectural components, technology choices, and deployment strategy. Each decision was made to balance development speed (hackathon constraints), maintainability, and the ability to demonstrate AI integration effectively.

Type	Details
System	Web Application (Next.js)
Architecture	Monolithic full-stack with server-side API routes
Database	SQLite via Prisma ORM
AI Service	Z AI GLM via z-ai-web-dev-sdk
Deployment	Standalone production build with Bun runtime

StockWise is a single-page application built with Next.js App Router, leveraging React 19 for the UI layer and Tailwind CSS 4 for styling. The frontend features a dark theme with emerald green accents, providing a modern and professional interface optimized for inventory management workflows. The UI components are built using shadcn/ui, a collection of reusable, accessible React components that ensure consistent design language across all pages.

All data flows through server-side API routes following REST conventions. This architecture ensures that sensitive operations, such as database queries and AI API calls, are executed on the server, keeping API keys secure and reducing the attack surface. The SQLite database is managed via Prisma ORM, which provides type-safe database access, automatic schema migrations, and a seed script (`prisma/seed.ts`) that populates the database with realistic demo data including products, suppliers, inventory records, sales history, and exchange rates.

Z AI GLM is integrated server-side via the z-ai-web-dev-sdk package. This integration is encapsulated within dedicated API route handlers, keeping the AI logic separate from the UI layer. The GLM is used for two primary functions:

(1) generating comprehensive inventory recommendations that consider multiple data dimensions, and

(2) providing multi-factor trade-off analysis for strategy simulation scenarios. Both functions return structured JSON responses that are parsed, validated, and persisted to the database before being displayed to the user.

2.1.2 LLM as Service Layer

The GLM (General Language Model) is integrated as a dedicated service layer within Next.js API routes, acting as an intelligent reasoning engine that augments traditional rule-based inventory logic. Rather than simply displaying raw data, StockWise leverages the GLM to perform holistic analysis that considers multiple concurrent factors, which are inventory levels, sales velocity, supplier lead times, currency exchange rates, and market news, to produce contextually rich, actionable recommendations.

Two primary API endpoints invoke the GLM, each serving a distinct analytical purpose:

(1) Endpoint 1: POST /api/recommendations/generate
This endpoint collects comprehensive inventory data including all products with their associated supplier information, current stock levels, 30-day sales aggregates, current exchange rates, and recent market news articles. It then constructs a detailed prompt of approximately 2,000 tokens that instructs the GLM to perform a holistic analysis of each product's inventory position. The prompt includes explicit instructions for the response format (JSON array), analysis criteria (urgency, cost impact, market alignment), and the requirement for transparent reasoning chains. The GLM's JSON array response is parsed, each recommendation's SKU is validated against the database, and the results are saved with full reasoning chains and news references for user review.

(2) Endpoint 2: POST /api/simulate
This endpoint implements a two-phase analysis approach. In the first phase, it runs a mathematical simulation that calculates days of stock remaining for each product using the formula: current stock minus projected sales equals days left. In the second phase, it invokes the GLM with the mathematical simulation results combined with exchange rate data and market news. The GLM then generates a multi-factor trade-off analysis that compares at least three different scenarios (e.g., aggressive restocking vs. conservative approach vs. market-timed purchasing), helping business owners understand the implications of each strategy before committing resources.

2.1.3 Dependency Diagram

The following describes the system dependency architecture. Since this is a text-based document, the relationships are presented using structured notation:

Core Data Flow:
Frontend (React SPA) → API Routes (Next.js Server) → Prisma ORM → SQLite Database

AI Integration Flow:
Frontend → API Routes → Z AI GLM (via z-ai-web-dev-sdk) → JSON Response → Database

Component Dependencies:

- React Components depend on TanStack Query for data fetching and caching
- API Routes depend on Prisma Client for database operations
- AI Endpoints depend on z-ai-web-dev-sdk for GLM communication
- Database Schema depends on Prisma Migrate for version control
- Demo Data depends on Prisma Seed script for initial population

Detailed Data Flow for AI Recommendations:

Step	Description
1	Frontend triggers a POST request to /api/recommendations/generate.
2	The API route queries the database for Products (with Supplier and Inventory relations), Sales records (30-day aggregates with daily averages and revenue calculations), and current ExchangeRates.
3	Market news is loaded from a server-side constant array containing 6 articles covering RM/USD exchange rate trends, supply chain disruptions, and semiconductor demand forecasts.
4	All collected data is formatted into a structured prompt with a system message defining the GLM's role as an inventory optimization analyst.
5	The z-ai-web-dev-sdk sends the constructed prompt to the GLM API endpoint.
6	The GLM returns a JSON array containing individual recommendation objects.
7	The API route parses the JSON response, validates that each SKU exists in the database, and saves the recommendations with full reasoning chains and news

	references.
8	The frontend displays the recommendations with expandable reasoning details and clickable news reference badges.

2.2. Technological Stack

The technology stack for StockWise was carefully selected to maximize development velocity during the hackathon while ensuring a robust, maintainable codebase. The following table provides a comprehensive breakdown of the technologies used across each layer of the application.

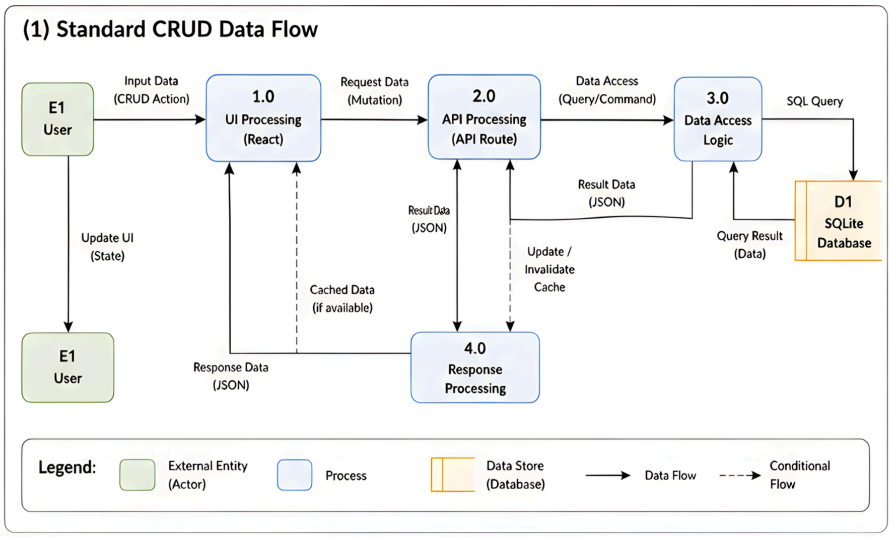
Layer	Technology	Purpose
Frontend	Next.js 16	Full-stack React framework with App Router, server components, and API routes
Frontend	React 19	UI component library with hooks for state management and side effects
Frontend	Tailwind CSS 4	Utility-first CSS framework for rapid, consistent styling with dark theme support
Frontend	shadcn/ui	Pre-built, accessible UI component library (buttons, cards, dialogs, tables, etc.)
Frontend	Recharts	Charting library for dashboard visualizations (bar charts, line charts, pie charts)
Frontend	Lucide React	Icon library providing consistent, scalable SVG icons throughout the interface
Frontend	TanStack Query	Server state management for data fetching, caching, and automatic refetching
Backend	Next.js API Routes	Server-side REST API endpoints for all data operations and AI integration
Backend	Prisma ORM	Type-safe database client with schema definitions, migrations, and seed scripts
Backend	z-ai-web-dev-sdk	Official SDK for communicating with Z AI GLM API for AI-powered analysis
Database	SQLite (better-sqlite3)	Embedded relational database providing zero-configuration local persistence
Deployment	Bun Runtime	High-performance JavaScript runtime for running the production build
Deployment	Standalone Build	Next.js standalone output for optimized, self-contained production deployment

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

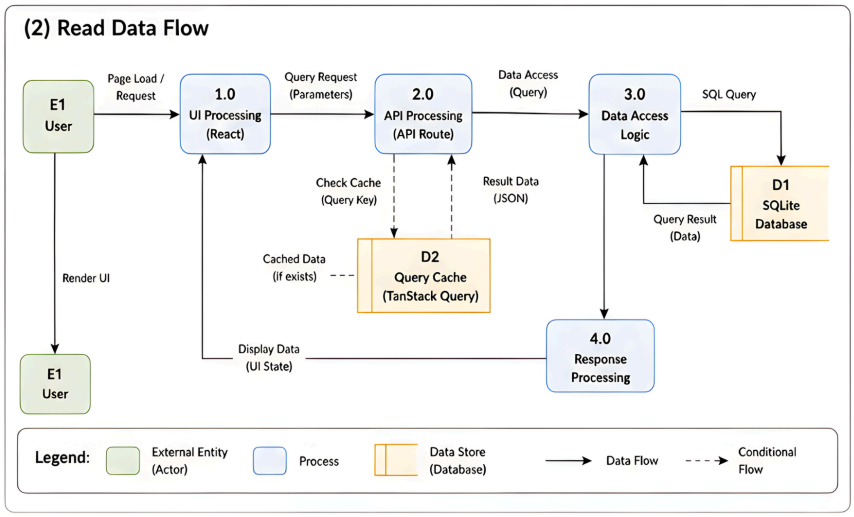
The data flow architecture of StockWise follows a unidirectional pattern that ensures predictable state management and clear separation of concerns between the presentation layer, business logic layer, and data persistence layer.

(1) Standard CRUD Data Flow:



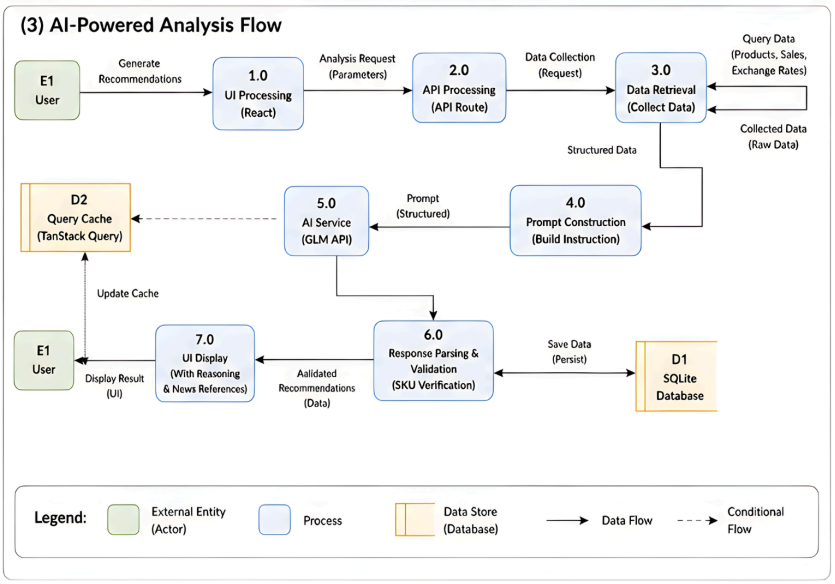
This diagram illustrates the standard CRUD operation flow within the system. User input is first processed by the UI layer, which sends a request to the API processing unit. The API interacts with the data access layer to perform database operations. The retrieved or updated data is then returned as a response and rendered back to the user interface. A cache mechanism is included to optimize performance by reducing redundant database queries.

(2) Read Data Flow:



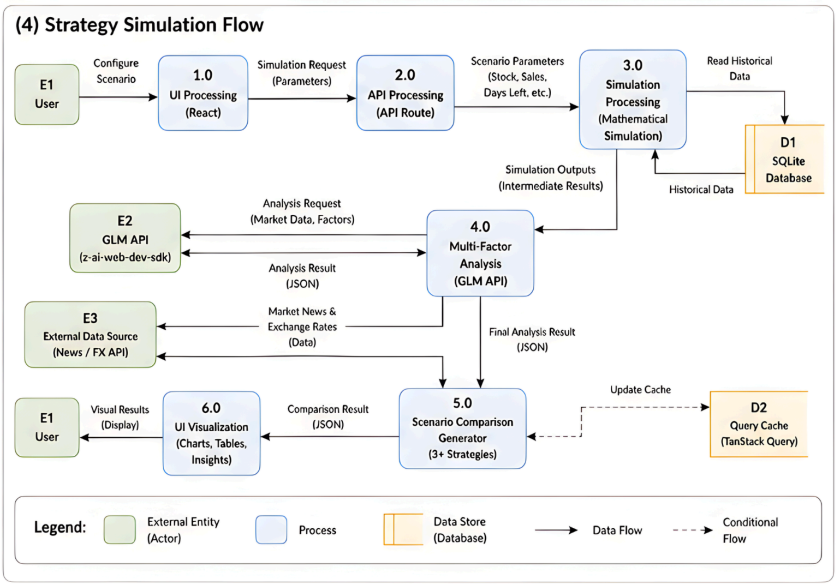
This diagram represents the data retrieval process triggered during page load or user requests. The system first checks the cache for existing data. If cached data is available, it is returned directly to the UI; otherwise, the request is forwarded to the database via the API. The final response is then displayed to the user.

(3) AI-Powered Analysis Flow:



This diagram describes the AI-driven analysis process. User requests are processed through the API, which gathers relevant data from the database. The collected data is then sent to an external AI service for analysis. The results are validated, stored, and subsequently presented to the user through the UI.

(4) Strategy Simulation Flow:



This diagram illustrates the simulation process for strategy evaluation. User-defined parameters are sent to the system for simulation processing. The system integrates historical data and external market information, and may utilize AI-based analysis to generate insights. The final results are visualized and presented to the user.

2.3.2. Database Schema

The StockWise database schema consists of 7 interconnected models designed through Prisma ORM. The schema enforces referential integrity through foreign key constraints and includes appropriate indexes for query performance. Below is a detailed description of each model.

Model	Fields	Description
Supplier	id (Int, PK, auto-increment), name (String), country (String), currency (String)	Represents product suppliers with their geographic location and operating currency. Used to determine exchange rate applicability and supplier diversity analysis.
Product	id (Int, PK), sku (String, unique), name (String), category (String), unitPriceMyr (Float), reorderPoint (Int), leadTimeDays (Int), supplierId (Int, FK → Supplier)	Core product catalog. Each product is linked to a single supplier. The reorderPoint and leadTimeDays fields are critical inputs for AI recommendation generation.
Inventory	id (Int, PK), productId (Int, FK → Product, unique), quantity (Int)	Tracks current stock levels. The unique constraint on productId ensures a 1:1 relationship with Product. Quantity is used to calculate stock status (Critical/Low/Healthy) based on reorderPoint thresholds.
Sale	id (Int, PK), productId (Int, FK → Product), quantity (Int), unitPrice (Float), total (Float), date (DateTime)	Records individual sales transactions. Aggregated over 30-day windows to calculate sales velocity metrics used by the AI model. Total is computed as quantity × unitPrice.
Recommendation	id (Int, PK), productId (Int, FK → Product), type (String), priority (String), message (String), reasoning (String),	Stores AI-generated recommendations. The reasoning field contains the GLM's analysis chain. newsReferences is a JSON string of related market articles.

	newsReferences (String, JSON), status (String), savingsMyr (Float)	Status tracks implementation state (pending/applied/dismissed).
Inventory Movement	id (Int, PK), productId (Int), type (String), quantity (Int), note (String)	Audit log for inventory changes. Records stock additions, removals, and adjustments with descriptive notes. Essential for tracking inventory history and debugging discrepancies.
Exchange Rate	id (Int, PK), fromCurrency (String), toCurrency (String), rate (Float)	Stores currency exchange rates (e.g., USD to MYR) used by the AI model to factor procurement cost implications into recommendations. Rates can be manually updated.

3. Functional Requirements & Scope

3.1. Minimum Viable Product

The Minimum Viable Product (MVP) for StockWise encompasses five core feature areas, each designed to demonstrate the practical value of AI integration in inventory management. These features were prioritized based on their impact on business decision-making and their suitability for hackathon demonstration.

#	Feature	Description
1	AI-Powered Recommendations	The core differentiating feature of StockWise. The system collects comprehensive inventory data (product details, current stock levels, supplier information), sales aggregates (30-day velocity, revenue, daily averages), currency exchange rates, and market news articles. All data is formatted into a structured prompt and sent to Z AI GLM, which performs a holistic multi-factor analysis. The GLM returns prioritized recommendations with full reasoning chains explaining the analysis logic, news references linking to relevant market articles, estimated cost savings in MYR, and actionable next steps. Each recommendation is tagged with a type (e.g., 'restock', 'diversify_supplier', 'hold') and priority level (critical, high, medium, low).
2	Strategy Validation & Simulation	Enables business owners to compare multiple procurement strategies before committing resources. The system first runs a mathematical simulation calculating days of stock remaining using current inventory minus projected sales. It then invokes the GLM with simulation results, exchange rate data, and market news to generate a multi-factor trade-off analysis. The GLM produces at least three scenario comparisons (e.g., aggressive restocking, conservative hold, market-timed purchasing) with detailed pros/cons for each approach, helping decision-makers understand the risk-reward profile of each strategy.
3	Dashboard KPIs	The main dashboard provides a real-time overview of inventory health through key performance

		indicators. KPIs include total inventory value (sum of all stock at unit price), number of products at critical or low stock levels, count of pending AI-generated suggestions requiring review, and estimated savings from AI recommendations that have been applied. The dashboard uses Recharts for interactive visualizations including bar charts for stock distribution, line charts for sales trends, and alert badges for items requiring immediate attention.
4	Inventory Management	Full CRUD capabilities for inventory tracking. Each product's stock status is automatically calculated by comparing current quantity against the defined reorder point: Critical (quantity ≤ 25% of reorder point), Low (quantity ≤ reorder point), or Healthy (quantity > reorder point). The interface supports manual stock adjustments with descriptive notes, all of which are logged in the Inventory Movement table for audit purposes. Users can filter and search inventory by product name, SKU, category, or stock status.
5	Sales Velocity Tracking	Comprehensive sales monitoring and logging capabilities. The system tracks individual sales transactions with product selection, quantity, unit price, and automatic total calculation. Sales data is aggregated to compute velocity metrics including 30-day total revenue, daily average sales, and average order value. These metrics feed directly into the AI recommendation engine and are displayed through interactive charts that help identify sales trends and seasonal patterns.

3.2. Non-Functional Requirements (NFRs)

Non-functional requirements define the quality attributes and operational constraints of the system. These requirements ensure that StockWise not only functions correctly but also provides a reliable, efficient, and user-friendly experience.

Quality Attribute	Requirement	Implementation
Token Latency	AI API calls must respond within 15 seconds to maintain user	Configurable timeout on z-ai-web-dev-sdk calls. Frontend

	engagement. The system must handle timeouts gracefully without crashing or leaving the UI in an inconsistent state.	displays loading spinners and progress indicators during AI processing. Error boundaries catch and display user-friendly error messages if the API call fails or times out.
Cost Efficiency	Minimize AI API costs through intelligent rate limiting and prompt optimization. The system should not exceed 10 AI API calls per hour under normal usage patterns.	Rate limiting implemented at the API route level (10 calls/hour). Prompt size carefully managed at approximately 2,000 tokens to balance analysis depth with cost efficiency. TanStack Query caching reduces redundant data fetches.
Reliability	Data must persist reliably across server restarts. The system should not lose inventory records, sales history, or AI recommendations due to process termination or crashes.	SQLite database provides durable, file-based persistence that survives server restarts. WAL (Write-Ahead Logging) mode enabled for better concurrent read performance. Prisma ORM ensures atomic transactions for data integrity.
Maintainability	The codebase must be structured for easy maintenance and feature extension. New developers should be able to understand the architecture within a reasonable onboarding period.	Modular API route organization with clear separation of concerns. Shared data-fetching hooks via TanStack Query ensure consistent data access patterns. Prisma schema serves as the single source of truth for data models. Environment variables for all configurable values.
Usability	The interface must be intuitive, responsive, and accessible across desktop and mobile devices. Dark theme should reduce eye strain during extended monitoring sessions.	Dark theme with emerald green accents provides a modern, professional appearance. Responsive design using Tailwind CSS breakpoints adapts layout for all screen sizes. Mobile-optimized bottom navigation bar provides quick

		access to all sections. Quick action buttons on the dashboard enable one-click access to common tasks.

3.3. Out of Scope / Future Enhancements

The following features and capabilities are explicitly excluded from the MVP scope of StockWise. These items represent potential future enhancements that would require additional development time, infrastructure, or third-party service integrations beyond the hackathon timeframe.

- Real-time news scraping: The current implementation uses a static array of market news articles. A production version would require integration with news APIs or RSS feeds for real-time market intelligence updates.
- Multi-user authentication and authorization: The MVP operates as a single-user application without login, roles, or permissions. A production deployment would need user management with JWT or session-based authentication.
- Automated purchase order (PO) generation: While the AI recommends procurement actions, the system does not automatically generate or send purchase orders to suppliers. This would require supplier portal integration.
- Native mobile applications: The system is designed as a responsive web application. Native iOS and Android apps would require separate development using React Native or similar frameworks.
- ERP system integration: The MVP does not integrate with external ERP systems (e.g., SAP, Oracle). Such integration would require standardized API connectors and data mapping layers.
- WebSocket real-time updates: The current implementation uses polling (TanStack Query refetch intervals) for data updates. WebSocket connections for real-time push notifications are deferred to a future version.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

For the MVP phase developed during UMHackathon 2026, the primary evaluation method is manual testing conducted by the development team. This includes functional testing of all API endpoints, UI testing across different browsers and screen sizes, and validation of AI-generated recommendations for accuracy, relevance, and actionable quality. Each feature is tested against its acceptance criteria as defined in Section 3 of this document.

For future production deployment, the team recommends implementing A/B testing frameworks to evaluate the accuracy and business impact of AI recommendations. This would involve comparing inventory outcomes when following AI suggestions versus manual decision-making, measuring metrics such as stockout reduction, carrying cost savings, and revenue impact over defined evaluation periods.

In the event of critical failures or unexpected behavior, the deployment strategy includes the ability to revert to a previous stable build using the deployment script. This rollback mechanism ensures that the system can be quickly restored to a known-good state while issues are investigated and resolved. Additionally, all database schema changes are managed through Prisma Migrate, providing version-controlled rollback capabilities for the data layer.

4.2. Monitoring

The current monitoring approach for StockWise is designed for the hackathon MVP environment and focuses on basic observability through server-side logging. Server logs are captured using the `tee` command to simultaneously output to the console and append to timestamped log files, enabling post-incident analysis.

Key monitoring areas include: API error tracking (logging all HTTP 4xx and 5xx responses with request details and stack traces), AI service response monitoring (tracking GLM API call durations, success rates, and response quality), rate limit enforcement (logging when users approach or exceed the 10 calls/hour threshold), and database query performance (identifying slow queries that may need optimization).

For production deployment, the team recommends upgrading to structured logging with JSON-formatted log entries, implementing health check endpoints (`/api/health`) for uptime monitoring services, and potentially integrating with observability platforms such as Grafana or Datadog for real-time dashboards and alerting capabilities.

4.3. Assumptions

The development and operation of StockWise is based on the following assumptions, which should be validated before production deployment:

- Users will access the system using modern web browsers (Chrome, Firefox, Safari, Edge) with JavaScript enabled. The application does not support Internet Explorer or browsers with JavaScript disabled.
- A reliable internet connection is available for accessing both the web application and the Z AI GLM API. The system does not currently support offline mode or local-first data synchronization.
- The application will be deployed on a single server instance. The current architecture does not include horizontal scaling, load balancing, or distributed session management.
- Demo data seeded via the Prisma seed script (prisma/seed.ts) is representative of real-world inventory patterns. The seed script populates 8 products across 4 suppliers with 90 days of simulated sales history.
- The Z AI GLM API will maintain consistent response formats and latency characteristics. Prompt engineering has been optimized for the current model version, and model upgrades may require prompt adjustments.
- A single user or a small team will use the system concurrently. The SQLite database is not optimized for high-concurrency write scenarios that would be expected in large-scale

4.4. External Dependencies

StockWise relies on several external tools and services for its core functionality. The following table identifies each dependency, its purpose, associated risks, and mitigation strategies.

Tool / Service	Purpose	Risks & Mitigation
Z AI GLM API (via z-ai-web-dev-sdk)	Core AI reasoning engine for generating inventory recommendations, performing strategy simulations, and providing multi-factor trade-off analysis with transparent reasoning chains.	Risk: API rate limits may restrict usage during peak demand. Service downtime could prevent recommendation generation. Response format changes could break JSON parsing. Mitigation: Graceful error messages displayed to users when API is unavailable. Rate limiting enforced at application level. Response validation before database persistence.

SQLite (via better-sqlite3 and Prisma ORM)	Embedded relational database providing zero-configuration, file-based data persistence for all application data including products, inventory, sales, recommendations, and exchange rates.	Risk: Not suitable for high-concurrency write operations or multi-server deployments. Limited support for advanced SQL features. Single-file database is vulnerable to corruption if the process crashes during write operations. Mitigation: Acceptable limitation for MVP and demo purposes. WAL mode enabled for better read concurrency. Regular database file backups recommended.
Bun Runtime	High-performance JavaScript runtime used to execute the Next.js production build. Bun offers faster startup times and lower memory usage compared to Node.js.	Risk: Potential compatibility issues with certain npm packages that rely on Node.js-specific APIs. Smaller community and ecosystem compared to Node.js. Mitigation: The application uses well-supported packages. Fallback to Node.js is straightforward if compatibility issues arise.
Next.js (App Router)	Full-stack React framework providing server-side rendering, API routes, and the App Router architecture that forms the foundation of the entire application.	Risk: Rapid version updates may introduce breaking changes. App Router patterns are still evolving. Mitigation: Pin exact version in package.json. Thorough testing after any framework upgrades.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline

The StockWise project was developed over a 10-day period in preparation for UMHackathon 2026. The timeline was structured to progressively build functionality, starting with foundational infrastructure and culminating in the AI integration that defines the system's core value proposition.

Day(s)	Phase	Activities & Deliverables
Day 1–2	Project Setup & Foundation	Initialized Next.js 16 project with TypeScript configuration. Set up Prisma ORM with SQLite database. Designed and implemented the complete database schema (7 models). Created the Prisma seed script with realistic demo data (8 products, 4 suppliers, 90 days of sales history). Configured Tailwind CSS 4 with dark theme and emerald accent color system. Integrated shadcn/ui component library.
Day 3–4	Core Pages Development	Built the main Dashboard page with KPI cards and summary statistics. Implemented the Inventory Management page with stock level display, status indicators (Critical/Low/Healthy), and manual adjustment functionality. Developed the Products catalog page with supplier information display. Created the shared layout with responsive sidebar navigation.
Day 5–6	Extended Features	Implemented the Sales Velocity page with transaction logging and revenue charts. Built the Exchange Rates management page for viewing and updating currency conversion rates. Created the Market News page displaying relevant industry articles. Added Recharts visualizations across all pages for data-driven insights.
Day 7–8	AI Recommendations & GLM Integration	Integrated z-ai-web-dev-sdk into the Next.js API route layer. Implemented the /api/recommendations/generate endpoint with

		comprehensive prompt engineering. Developed the AI Recommendations display page with reasoning chain expansion and news reference badges. Tested and refined prompt construction for optimal GLM response quality and JSON formatting.
Day 9	Strategy Validation	Implemented the mathematical simulation engine (stock projection calculations). Built the /api/simulate endpoint combining mathematical results with GLM multi-factor analysis. Developed the Strategy Simulation page with scenario comparison display and trade-off analysis visualization. Tested scenario generation for consistency and actionability.
Day 10	Polish, Testing & Bug Fixes	Conducted comprehensive end-to-end testing of all features. Fixed UI inconsistencies and responsive design issues. Optimized loading states and error handling across all AI-powered features. Finalized documentation and prepared demo presentation. Performed final database seed verification and data integrity checks.

5.2. **Team Members Role**

StockWise was developed by a small, focused team of 4 members with complementary skill sets covering frontend development, backend engineering, and AI integration. The team adopted an agile development approach with daily stand-ups and iterative feature delivery.

The frontend development focused on building responsive, user-friendly interfaces using React 19, Tailwind CSS 4, and shadcn/ui components. This included implementing the dashboard KPIs, inventory management pages, sales tracking, and all data visualization components using Recharts. Special attention was given to the dark theme implementation with emerald accents, ensuring consistent visual design across all pages and mobile-responsive layouts with bottom navigation.

The backend engineering efforts centered on designing and implementing the database schema through Prisma ORM, building all server-side API routes for CRUD operations, and ensuring data integrity through proper validation and error handling. This also included creating the seed

script for demo data generation and implementing the mathematical simulation engine for strategy validation.

The AI integration work involved designing the prompt engineering strategy for both the recommendation generation and strategy simulation endpoints, implementing the z-ai-web-dev-sdk communication layer, developing robust JSON response parsing with validation, and ensuring that AI-generated content (reasoning chains, news references, savings estimates) was properly stored and displayed to users. This role required close collaboration between all team members to ensure the AI outputs aligned with the UI design and business requirements.

In addition to the core development roles, a dedicated system integration and quality assurance role was established to ensure overall system reliability and cohesion. This role focused on coordinating the integration between frontend, backend, and AI components, verifying that data flows correctly across the full stack from user input to AI-generated output. It also involved conducting end-to-end testing, validating API responses, and ensuring that AI recommendations are displayed accurately with proper reasoning and impact metrics. Particular attention was given to handling edge cases such as incomplete data, API failures, and delayed responses, ensuring that the system maintains stability and provides meaningful feedback to users under all conditions.

5.3. Recommendations

Based on the lessons learned during development and the feedback received during UMHackathon 2026, the team recommends the following enhancements for taking StockWise from an MVP to a production-ready inventory management platform:

(1) Add Redis for caching repeated AI queries: Since AI recommendations for the same inventory state do not change frequently, implementing a Redis caching layer would significantly reduce API costs and improve response times. Cache entries could be invalidated when inventory or sales data changes.

(2) Migrate to PostgreSQL for production deployment: While SQLite is excellent for development and demo purposes, a production deployment handling multiple concurrent users would benefit from PostgreSQL's superior concurrency handling, advanced querying capabilities, and established backup/recovery tooling.

(3) Implement real-time news RSS feed integration: Replace the static market news array with automated RSS feed parsing or news API integration (e.g., Bloomberg, Reuters) to provide continuously updated market intelligence. This would significantly enhance the timeliness and relevance of AI recommendations.

(4) Add user authentication and role-based access control: Implement JWT-based authentication with role definitions (Admin, Warehouse Manager, Procurement Officer) to support multi-user scenarios. Each role would have tailored dashboard views and permission levels appropriate to their responsibilities.

(5) Integrate automated purchase order generation: Extend the AI recommendation system to automatically draft purchase orders based on accepted recommendations, with supplier-specific formatting and direct email dispatch capabilities.

(6) Develop a mobile companion app: Build a React Native mobile application providing push notifications for critical stock alerts, quick inventory checks, and approval workflows for AI-generated recommendations while on the warehouse floor.